MODULE 2

PYTHON PROGRAMMING

1BPLC105B / 205B

STRINGS • TUPLES • LISTS • DATA STRUCTURES

Course Notes

Kiran Mudaraddi

Understand the concept • Trace the code • Check the output • Practise

MODULE II — CONTENTS

- PART A — STRINGS
- 1. Strings as single things and parts
- 2. Length, traversal and for loop
- 3. Slices and string comparison
- 4. Immutability; in and not in
- 5. User-defined find function and built-in find()
- 6. Looping and counting; optional parameters
- 7. split(), cleaning strings and format()
- PART B — TUPLES
- 8. Grouping data; tuple creation and access
- 9. Tuple assignment and swapping
- 10. Tuples as return values
- 11. Composability and tuple operations
- PART C — LISTS
- 12. List values, access, length and membership
- 13. List operations, slices, mutability and deletion
- 14. Objects, references, aliasing and cloning
- 15. Lists and loops; list parameters and methods
- 16. Pure functions, modifiers and list-producing functions
- 17. Strings/lists, list() and range(), nested lists and matrices
- PART D — PRACTICE & REVISION
- 18. Worked programs, debugging checklist and practice questions

MODULE II — LEARNING OUTCOMES

After studying this module, students should be able to:

- Explain strings as sequences and use indexing, slicing, traversal and comparison.
- Use string membership, searching, counting, optional parameters, split(), strip() and format().
- Create tuples, unpack values, swap variables and return multiple values from a function.
- Explain lists as mutable sequences and perform list operations and slicing.
- Distinguish aliasing from cloning and explain how functions may modify lists.
- Write programs using list methods, functions that produce lists, nested lists and matrices.

How to use these notes

Read the explanation first. Then run the example, compare the output, change one value, and run it again. Indentation in Python is part of the program.

PART A — STRINGS

1. Working with Strings as Single Things

A string is a sequence of characters written inside single or double quotes. A string can be stored in a variable, displayed, compared, searched, and processed.

Syntax: `variable_name = "text"`

In [ ] Python code

```
message = "Hello Python"
print(message)
```

Out [ ]

```
Hello Python
```

2. Working with Parts of a String

Individual characters are accessed by index. The first character has index 0. Negative indices count from the end.

In [ ] Python code

```
fruit = "Apple"
print(fruit[0])
print(fruit[3])
print(fruit[-1])
```

Out [ ]

```
A
l
e
```

Remember

For the string "Apple", valid positive indices are 0 through 4. Accessing an index outside the string raises an IndexError.

3. Length of a String

The built-in `len()` function returns the number of characters in a string.

In [ ] Python code

```
word = "Computer"
print(len(word))
```

Out [ ]

```
8
```

4. Traversal and the for Loop

Traversal means visiting each character in sequence. A `for` loop is a direct way to process every character.

In [ ] Python code

```
name = "BITM"
for ch in name:
    print(ch)
```

Out [ ]

```
B
I
T
M
```

5. Slices

A slice extracts part of a string. The start index is included and the end index is excluded.

In [ ] Python code

```
word = "Python"
print(word[0:4])
print(word[2:])
print(word[:3])
print(word[1:5:2])
```

Out [ ]

```
Pyth
thon
Pyt
yh
```

Expression	Meaning	Result for "Python"
word[0:4]	Index 0 up to, but not including, 4	Pyth
word[2:]	Index 2 to the end	thon
word[:3]	Beginning through index 2	Pyt
word[::2]	Every second character	Pto

6. String Comparison

Strings can be compared using `==`, `!=`, `<`, `>`, `<=`, `>=`. Comparison is lexicographic and depends on character code values; it is case-sensitive.

In [ ] Python code

```
print("apple" == "apple")
print("Apple" < "banana")
print("Python" == "python")
```

Out [ ]

```
True
True
False
```

> **Note**
> Do not interpret string comparison as dictionary order in every language. In Python, uppercase and lowercase letters have different code points.

7. Strings are Immutable

Strings are immutable: an individual character cannot be changed in place. Build a new string when a different value is needed.

In [ ] Python code

```
text = "Hello"
# text[0] = "J"  # TypeError if uncommented
new_text = "J" + text[1:]
print(new_text)
```

Out [ ]

```
Jello
```

8. The in and not in Operators

The `in` operator checks whether a character or substring is present. `not in` checks absence.

In [ ] Python code

```
fruit = "banana"
print("a" in fruit)
print("z" not in fruit)
print("nan" in fruit)
```

Out [ ]

```
True
True
True
```

9. A User-Defined find Function

A user-defined function can scan the string from left to right. It returns the index of the first match, or -1 if no match is found.

In [ ] Python code

```
def find_letter(word, letter):
    index = 0
    while index < len(word):
        if word[index] == letter:
            return index
        index += 1
    return -1

print(find_letter("banana", "n"))
print(find_letter("banana", "z"))
```

Out [ ]

```
2
-1
```

10. Looping and Counting

A counter is initialized before the loop and increased whenever the condition is satisfied.

In [ ] Python code

```
word = "banana"
count = 0
for ch in word:
    if ch == "a":
        count += 1
print("Count:", count)
```

Out [ ]

```
Count: 3
```

11. Optional Parameters

A parameter can have a default value. If the caller omits that argument, Python uses the default.

In [ ] Python code

```
def greet(name, msg="Good Morning"):
    print(msg, name)

greet("Sam")
greet("Kiran", "Welcome")
```

Out [ ]

```
Good Morning Sam
Welcome Kiran
```

Remember

Parameters with default values are optional when the function is called. In a function definition, parameters without defaults must come before parameters with defaults.

12. The Built-in find() Method

The string method `find()` returns the lowest index where the substring occurs, or -1 when it is not found.

In [ ] Python code

```
text = "Learning Python"
print(text.find("Python"))
print(text.find("java"))
print("banana".find("a", 2))
```

Out [ ]

```
9
-1
3
```

13. The split() Method

The `split()` method divides a string into a list of substrings. With no argument, it splits on runs of whitespace.

In [ ] Python code

```
line = "Welcome to BITM Ballari"
words = line.split()
print(words)
data = "2025-10-28"
print(data.split("-"))
```

Out [ ]

```
['Welcome', 'to', 'BITM', 'Ballari']
['2025', '10', '28']
```

14. Cleaning Up Strings

Whitespace at the beginning or end of text can be removed with `strip()`, `lstrip()`, and `rstrip()`.

In [ ] Python code

```
text = "   Hello BITM   "
print("[" + text.strip() + "]")
print("[" + text.lstrip() + "]")
print("[" + text.rstrip() + "]")
```

Out [ ]

```
[Hello BITM]
[Hello BITM   ]
[   Hello BITM]
```

15. The format() Method

The `format()` method inserts values into placeholders and can control how numbers are displayed.

In [ ] Python code

```
name = "Kiran"
age = 35
print("My name is {} and I am {} years old.".format(name, age))
print("The value of pi is {:.2f}".format(3.14159))
```

Out [ ]

```
My name is Kiran and I am 35 years old.
The value of pi is 3.14
```

String quick revision

Concept	Example	Result
Length	len("abc")	3
Membership	"a" in "apple"	True
Built-in search	"apple".find("p")	1
Split	"bitm ballari".split()	['bitm', 'ballari']
Formatting	"Hello {}".format("World")	Hello World

PART B — TUPLES

8. Tuples: Grouping Data

A tuple is an ordered collection of values. It can group related values, including values of different types. Tuples are immutable and are commonly written with parentheses.

In [ ] Python code

```
student = ("Kiran", 35, "Ballari")
print(student)
empty_tuple = ()
one_item = (5,)
print(type(empty_tuple).__name__)
print(type(one_item).__name__)
```

Out [ ]

```
('Kiran', 35, 'Ballari')
tuple
tuple
```

Important

The comma makes a one-element tuple: `(5,)`. `(5)` is just the integer 5 inside parentheses.

9. Tuple Indexing and Access

Tuple items can be accessed by index, much like string characters and list elements.

In [ ] Python code

```
data = ("BITM", 2025, "CSE")
print(data[0])
print(data[2])
```

Out [ ]

```
BITM
CSE
```

10. Tuple Assignment (Unpacking)

Tuple unpacking assigns the elements of an iterable to multiple variables in one statement. The number of variables must match the number of unpacked values.

In [ ] Python code

```
(name, age, place) = ("Leo", 30, "Ballari")
print(name)
print(age)
print(place)
```

Out [ ]

```
Leo
30
Ballari
```

Swapping values

In [ ] Python code

```
a, b = 10, 20
a, b = b, a
print(a, b)
```

Out [ ]

```
20 10
```

11. Tuples as Return Values

A function can return multiple values separated by commas. Python packs those values into a tuple, which can then be unpacked by the caller.

In [ ] Python code

```
def divide(a, b):
    quotient = a // b
    remainder = a % b
    return quotient, remainder

q, r = divide(10, 3)
print("Quotient:", q)
print("Remainder:", r)
```

Out [ ]

```
Quotient: 3
Remainder: 1
```

Trace the result

For 10 divided by 3, integer division gives quotient 3 and the remainder operator gives 1. This example assumes the divisor is not zero.

12. Composability of Data Structures

Composability means combining data structures. A tuple can be placed inside a list, a list can be placed inside a tuple, and tuples can be nested.

In [ ] Python code

```
students = [("Kiran", 35), ("Vanita", 30), ("Samarth", 3)]
data = ("BITM", [2023, 2024, 2025])
record = ("CSE", ("AI", "ML", "DS"))
print(students[0])
print(data[1][2])
print(record[1][2])
```

Out [ ]

```
('Kiran', 35)
2025
DS
```

13. Tuple Operations

Operation	Example	Result
Concatenation	(1, 2) + (3, 4)	(1, 2, 3, 4)
Repetition	(10, 20) * 2	(10, 20, 10, 20)
Membership	3 in (1, 2, 3)	True
Length	len((5, 10, 15))	3

14. Immutability of Tuples

Tuple elements cannot be assigned, inserted or deleted directly. To represent a changed tuple, create a new tuple.

In [ ] Python code

```
t = (1, 2, 3)
t = t + (4,)
print(t)
```

Out [ ]

```
(1, 2, 3, 4)
```

Key points

Tuples are useful for fixed groups of values. A tuple can be a dictionary key only when all its elements are hashable; a tuple containing a list cannot be used as a dictionary key.

15. Tuple Practice Examples

In [ ] Python code

```
nums = (10, 14, 83, -9, 17)
print("Maximum:", max(nums))
print("Minimum:", min(nums))
```

Out [ ]

```
Maximum: 83
Minimum: -9
```

In [ ] Python code

```
def sum_and_average(values):
    total = sum(values)
    average = total / len(values)
    return total, average

num = (1, 2, 3, 4)
total, average = sum_and_average(num)
print("Sum:", total)
print("Average:", average)
```

Out [ ]

```
Sum: 10
Average: 2.5
```

Check before running

The average example expects a non-empty tuple. If the tuple is empty, len(values) is zero and division by zero would occur.

PART C — LISTS

16. List Values

A list is an ordered, mutable collection of items. Lists use square brackets and can contain values of different types.

In [ ] Python code

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

Out [ ]

```
['apple', 'banana', 'cherry']
```

17. Accessing Elements, Length and Membership

In [ ] Python code

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
print(fruits[-1])
print(len(fruits))
print("apple" in fruits)
print("grape" not in fruits)
```

Out [ ]

```
apple
cherry
3
True
True
```

18. List Operations

Lists support concatenation with + and repetition with *.

In [ ] Python code

```
a = [1, 2, 3]
b = [4, 5]
print(a + b)
print(a * 2)
```

Out [ ]

```
[1, 2, 3, 4, 5]
[1, 2, 3, 1, 2, 3]
```

19. List Slices

A list slice creates a new list containing the selected items. The start index is included and the end index is excluded.

In [ ] Python code

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[:3])
print(nums[::2])
```

Out [ ]

```
[20, 30, 40]
[10, 20, 30]
[10, 30, 50]
```

20. Lists are Mutable

Unlike strings and tuples, list elements can be reassigned after a list has been created.

In [ ] Python code

```
nums = [10, 20, 30, 40]
nums[2] = 99
print(nums)
```

Out [ ]

```
[10, 20, 99, 40]
```

21. List Deletion

In [ ] Python code

```
nums = [10, 20, 30, 40]
del nums[1]
print(nums)
```

Out [ ]

```
[10, 30, 40]
```

22. Objects and References

Assignment does not automatically clone a list. When one variable is assigned to another, both names can refer to the same list object.

In [ ] Python code

```
a = [1, 2, 3]
b = a
b[0] = 99
print(a)
print(b)
```

Out [ ]

```
[99, 2, 3]
[99, 2, 3]
```

23. Aliasing and Cloning

Aliasing occurs when multiple variables refer to the same list. A slice such as `a[:]` creates a shallow copy of the outer list.

In [ ] Python code

```
a = [1, 2, 3]
b = a[:]
b[0] = 99
print("a =", a)
print("b =", b)
```

Out [ ]

```
a = [1, 2, 3]
b = [99, 2, 3]
```

Remember

A slice-based clone is shallow. If a list contains other mutable lists, the inner lists are still shared between the original and the copy.

24. Lists and for Loops

In [ ] Python code

```
fruits = ["apple", "banana", "cherry"]
for item in fruits:
    print(item)
```

Out [ ]

```
apple
banana
cherry
```

25. List Parameters in Functions

A function receives a reference to the list object. If the function changes an element of that list, the change can be seen by the caller.

In [ ] Python code

```
def add_one(lst):
    for i in range(len(lst)):
        lst[i] += 1

values = [4, 5, 6]
add_one(values)
print(values)
```

Out [ ]

```
[5, 6, 7]
```

Terminology

It is more precise to say that Python passes object references by assignment. A function can mutate a list object, but assigning a new list to its local parameter does not rebind the caller's variable.

26. Common List Methods

Method	Purpose	Example
append(x)	Adds x to the end	nums.append(5)
sort()	Sorts the list in place	nums.sort()
reverse()	Reverses the list in place	nums.reverse()
remove(x)	Removes first matching value	nums.remove(3)
pop()	Removes and returns an item	nums.pop()
insert(i, x)	Inserts x at index i	nums.insert(1, 7)

In [ ] Python code

```
nums = [3, 1, 2]
nums.append(5)
print(nums)
nums.sort()
print(nums)
nums.reverse()
print(nums)
nums.remove(3)
print(nums)
```

Out [ ]

```
[3, 1, 2, 5]
[1, 2, 3, 5]
[5, 3, 2, 1]
[5, 2, 1]
```

27. Pure Functions and Modifiers

A pure-style function can return a new list without modifying the input. A modifier changes the list it receives.

In [ ] Python code

```
def pure_double(nums):
    return [n * 2 for n in nums]

def modify_double(nums):
    for i in range(len(nums)):
        nums[i] *= 2

values = [2, 3, 4]
new_values = pure_double(values)
print("Original:", values)
print("New list:", new_values)
modify_double(values)
print("After modifier:", values)
```

Out [ ]

```
Original: [2, 3, 4]
New list: [4, 6, 8]
After modifier: [4, 6, 8]
```

28. Functions that Produce Lists

In [ ] Python code

```
def squares(n):
    result = []
    for i in range(1, n + 1):
        result.append(i * i)
    return result

print(squares(5))
```

Out [ ]

```
[1, 4, 9, 16, 25]
```

29. Strings and Lists

The `list()` function can make a list of characters from a string. The `join()` string method combines strings from an iterable.

In [ ] Python code

```
letters = list("hello")
print(letters)
word = "".join(letters)
print(word)
print("-".join(["2025", "10", "28"]))
```

Out [ ]

```
['h', 'e', 'l', 'l', 'o']
hello
2025-10-28
```

30. list() and range()

In [ ] Python code

```
print(list(range(5)))
print(list(range(2, 7)))
print(list(range(2, 10, 2)))
```

Out [ ]

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6]
[2, 4, 6, 8]
```

31. Nested Lists and Matrices

A nested list contains lists as elements. A matrix can be represented as a list of rows. Use two indices to access an element.

In [ ] Python code

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]
print(matrix[1])
print(matrix[1][2])
```

Out [ ]

```
[4, 5, 6]
6
```

Traversing a matrix

In [ ] Python code

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
    for value in row:
        print(value, end=" ")
    print()
```

Out [ ]

```
1 2 3
4 5 6
7 8 9
```

PART D — PRACTICE & REVISION

32. Common Errors to Check

Error or misconception	What to remember
Using index 1 for the first item	Python indexing starts at 0.
Expecting the end of a slice to be included	The slice end index is excluded.
Changing a character in a string	Strings are immutable; construct a new string.
Writing (5) for a one-item tuple	Use (5,) to create a tuple.
Assuming b = a copies a list	It creates another reference to the same list.
Expecting sort() to return a sorted list	sort() modifies the list in place and returns None.
Using a shallow copy as a deep copy	Nested mutable elements may still be shared.

33. Short Practice Questions

1. Write a program to count the number of vowels in a string.

2. Write a function that returns the first index of a character, or -1 if it is absent.

3. Read a date in YYYY-MM-DD form and split it into year, month and day.

4. Create a tuple containing student name, age and branch. Print each value separately.

5. Swap two variables using tuple assignment.

6. Write a function that returns both the quotient and remainder of two integers.

7. Given a tuple of numbers, display the maximum and minimum.

8. Write a function that returns the sum and average of a non-empty tuple.

9. Create a list of five numbers and display its length, first item and last item.

10. Demonstrate list concatenation, repetition and slicing.

11. Show the difference between aliasing and cloning with a short program.

12. Write a function that adds one to every item in a list.

13. Write a function that returns squares from 1 to n.

14. Convert the string 'python' to a list of characters and join it back.

15. Create a 3 × 3 matrix and print the element in row index 1, column index 2.

34. Quick Revision Table

Data type	Ordered?	Mutable?	Typical syntax
String	Yes	No	"Python"
Tuple	Yes	No	(1, 2, 3)
List	Yes	Yes	[1, 2, 3]

Final revision strategy

For every example: predict the output, run the code, explain each line, then change one input and explain how the output changes.

Reference and scope

These notes follow the topics and classroom examples supplied in M-2_Strings.txt, M-2_Tuples.txt, M2-Lists.txt and TUPLES.txt. Concepts are arranged for student revision with explanations, runnable examples, outputs, practice questions and debugging reminders.

Recommended textbook: Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers, *How to Think Like a Computer Scientist: Learning with Python 3*, 3rd Edition (2020), particularly the chapters on strings, tuples and lists.